FaceTrack: Real-Time Face Recognition, Emotion Analysis, and Proportional Servo Tracking via Arduino USB Serial

Gayatri Dabhade, Achala Paradeshi, Akshat B Gupta, and Dr. S. D. Ruikar
Department of Electronics Engineering, Walchand College of Engineering, Sangli, India

***Abstract*—This paper presents FaceTrack, a real-time face detection, recognition, and servo-based camera tracking system implemented on a standard laptop with an Arduino Uno microcontroller. InsightFace (ArcFace, 512-dimensional embeddings) provides identity recognition via cosine similarity against a SQLite database. DeepFace classifies facial emotion in parallel. A proportional (P) controller operating on the pixel-space error signal replaces the direct pixel-to-angle mapping used in earlier prototypes, guaranteeing that the detected face is actively centred in the frame rather than tracked at a fixed angular offset. Exponential moving average (EMA) smoothing, a 25-pixel deadzone, and sub-degree suppression ensure jitter-free servo actuation over a USB serial link at 115,200 baud. A Flask web dashboard provides live MJPEG video, attendance management, alert generation, and GPT-4.1-nano-powered AI analytics. Testing with three enrolled subjects across indoor and outdoor conditions demonstrates recognition confidence of 61–83%, simultaneous detection of up to four faces, correct emotion labelling, and stable tracking sessions exceeding 18 minutes.**

***Keywords*—Face Recognition, ArcFace, Proportional Controller, Pan-Tilt Tracking, Arduino, InsightFace, DeepFace, Flask, Attendance System.**

I. Introduction

The convergence of real-time computer vision, embedded microcontroller systems, and cloud-connected AI has made it feasible to build intelligent face-tracking and attendance platforms on commodity hardware. FaceTrack demonstrates how a USB webcam, a standard Windows laptop, and an Arduino Uno microcontroller can be combined into a production-grade system that simultaneously detects and recognises faces, drives a physical pan-tilt camera mount, logs attendance with emotion data, and presents all information through a browser-based dashboard.

Prior work in the group (Project-I) used a Raspberry Pi 4 as the central processor, communicating servo commands over HTTP. Testing revealed three architectural deficiencies: InsightFace inference on ARM CPU achieved fewer than 5 fps; HTTP-over-Wi-Fi introduced 50–200 ms of unpredictable latency; and most critically, direct pixel-to-angle servo mapping caused the camera to settle at a fixed angular offset from the detected face rather than actively centring it — a fundamental control-loop error. FaceTrack resolves all three.

The principal technical contributions of this paper are: (1) identification and correction of the direct pixel-to-angle mapping error by replacing it with a proportional controller on the error signal; (2) a complete multi-threaded pipeline decoupling 30-fps capture from 12-fps detection; (3) integration of ArcFace recognition, DeepFace emotion analysis, SQLite attendance logging, and a GPT-4.1-nano AI assistant into a unified Flask application; and (4) a validated prototype achieving 61–83% recognition confidence and sub-15 ms servo latency on commodity hardware costing under ₹3,000.

II. Related Work

Viola and Jones [1] introduced Haar cascade classifiers for real-time CPU-based face detection. While widely deployed, they are sensitive to illumination changes and non-frontal poses. Google's MediaPipe BlazeFace [2] uses a lightweight single-shot detector optimised for CPU inference, achieving high detection accuracy at low latency without GPU acceleration. Deng et al. [3] introduced ArcFace, an additive angular margin loss for face recognition training that produces identity-discriminative 512-dimensional embeddings, achieving state-of-the-art accuracy on standard benchmarks. InsightFace provides production-ready ArcFace models.

Patil and Kulkarni [4] demonstrated servo-controlled face tracking by coupling MediaPipe with servo actuation via a Raspberry Pi GPIO. Their work used direct proportional mapping of face pixel coordinates to servo angles — the same architectural error identified in our Project-I prototype. A proportional-integral-derivative (PID) or P-only controller is the standard approach in embedded control: the control output should be proportional to the error signal, ensuring convergence to zero error, not to a fixed position. Serengil and Ozpinar [5] introduced the DeepFace library, providing a unified interface for facial attribute analysis including seven-class emotion classification. Shinde and Kulkarni [6] demonstrated Flask-based web dashboards for real-time vision monitoring. The AI assistant component of FaceTrack extends this with large language model (LLM)-powered briefing and anomaly detection.

III. System Architecture

FaceTrack separates concerns across four layers: (a) hardware input (USB camera at 640×480, 30 fps); (b) laptop-side vision and control (Python 3.10, Flask, InsightFace, DeepFace, EMA P-controller); (c) microcontroller servo driver (Arduino Uno via 115,200-baud USB serial); and (d) actuator (two SG90 servos, pan 40–140°, tilt 55–125°). Three concurrent Python threads prevent any single stage from blocking the others.

A. Multi-Thread Pipeline

Thread 1 (camera worker) captures frames at 30 fps, applies cv2.flip(frame, 1) for mirror-mode display, runs the draw loop, and encodes the annotated frame as MJPEG for the /video_feed endpoint. Thread 2 (detection worker) reads the latest raw frame from a lock-protected shared buffer and runs InsightFace at approximately 12 fps, performing IoU-based track matching to assign stable track IDs. Thread 3 (recognition worker) receives cropped face images via a queue, compares ArcFace embeddings against the SQLite database, and writes identity results

through a shared map. This architecture sustains a smooth 30-fps browser stream regardless of detection latency.

B. P-Controller Correction — Core Contribution

The fundamental error in the Project-I prototype was the control law:

```
servo_angle = f(face_pixel_x)
        [INCORRECT]
```

This maps face position to servo angle. If the face is at pixel x = 400 (80 px right of the 320-pixel frame centre), the servo moves to approximately 78° and holds there. The face remains 80 px off-centre indefinitely: the servo has locked on but has not centred the face.

The correct control law is:

```
error      = face_px − frame_centre_px
smoothed   = α × smoothed + (1−α) × error
new_angle = current_angle + Kp × smoothed
```

When the face reaches the frame centre, error → 0, Δangle → 0, and the servo holds still. This is self-correcting by construction. The EMA filter (Equation 2) with α ramping from 0.20 at acquisition to 0.65 at stable tracking over 45 frames smooths detection noise without adding significant lag.

Table I lists the tuned controller parameters.

TABLE I

P-Controller Parameters

Parameter	Value	Parameter	Value
Kp_PAN	0.12 °/px	Initial EMA α	0.20
Kp_TILT	0.10 °/px	Stable EMA α	0.65
Deadzone	25 px	Ramp frames	45 (~1.5 s)
Min Δangle	1.0°	Serial baud	115,200

C. Arduino Serial Protocol

Three ASCII commands are defined (terminated by \n): “pan,tilt\n” moves both servos to the specified integer angles; “centre\n” snaps both axes to 90°; “quench\n” detaches servos for power saving. The Arduino drains all available serial bytes on every loop iteration before applying a 1 ms yield delay, changed from the original 10 ms that caused command queuing and jitter. Received angles are written directly to the servos via Servo.write() without further smoothing on the Arduino side, since all EMA filtering is performed in Python.

D. Recognition and Attendance Pipeline

ArcFace embeddings (512-d) are extracted for each detected face and compared against all stored embeddings via cosine similarity. The enrolled person with the highest mean similarity above threshold 0.5 is identified. When a confirmed identity persists for 10 consecutive frames (IDENTITY_FLIP_THRESH), a check-in event is logged to SQLite if at least 60 seconds have elapsed since the last log. DeepFace analyses the aligned face crop and stores the dominant emotion (angry, disgust, fear, happy, neutral, sad, surprise) alongside the attendance record.

IV. Implementation

The system is implemented in Python 3.10+ on Windows 10/11. Key libraries: OpenCV 4.x (capture, draw, JPEG encode), InsightFace (ArcFace R100 backbone), DeepFace (emotion), Flask 2.x (web server, MJPEG, REST), pyserial 3.x (Arduino bridge), SQLAlchemy + SQLite (ORM), gTTS + pygame (non-blocking TTS), python-dotenv (configuration), and the OpenAI SDK via AIPipe (GPT-4.1-nano). The Arduino Uno runs a 47-line C++ sketch using Servo.h; no OS is required, and it boots and begins accepting serial commands in under one second.

The Flask application exposes REST endpoints for camera control (/camera/start, /camera/stop, /api/camera/switch), servo configuration (/api/servo/configure), tracking target selection (/api/track/<uid>, /api/track/stop), attendance queries (/api/attendance), alert management (/alerts, /alert/<id>/resolve), user enrolment and profiles (/users, /user/<id>), and AI assistant functions (/api/ai/chat, /api/ai/briefing, /api/ai/anomalies, /api/ai/person/<id>/summary).

Five known bugs were identified and corrected during development. The Arduino sketch originally applied an independent EMA (alpha = 0.20) to the received angles, compounding the Python-side EMA and introducing two full time-constants of latency; removing it brought the servo response into alignment with the commanded trajectory. The loop delay was reduced from 10 ms to 1 ms to prevent serial command accumulation. The Python EMA alpha values were inverted (INITIAL = 0.72, STABLE = 0.50, producing slow acquisition and fast stable tracking, opposite of the desired behaviour); these were corrected to INITIAL = 0.20, STABLE = 0.65. The MAX_SPEED pixel cap was raised from 100 px/s to 400 px/s (3.3 → 13 px/frame) to allow timely acquisition. The app.py rate-limiter timestamp was corrected to update on every interval tick regardless of whether the face was the active tracking target, preventing burst commands on target switches.

V. Experimental Results

The system was tested with three enrolled subjects (Akshat B Gupta, Gayatri, Achala) across indoor laboratory and outdoor conditions, varying distance (0.5–2 m), face angle (±30° off-axis), lighting (fluorescent, natural, backlit), and scene complexity (one to four simultaneous faces).

A. Recognition Performance

Recognition confidence ranged from 61% to 83% across the test sessions. Peak values (Akshat 79%, Gayatri 83%) were achieved under frontal lighting at approximately 1 m. The lowest values (Akshat 55.6–61%) occurred in backlit or high-angle conditions. No false-positive identity assignments were observed for any of the three unregistered subjects appearing in the test scenes. Achala was recognised at 80% confidence on an earlier build.

B. Multi-Face Detection and Prioritisation

Up to four simultaneous faces were detected correctly in a single frame, with the enrolled person

labelled PRIMARY and three unknown individuals assigned distinct track IDs (Unknown #1, #2, #3) using red bounding boxes. No false-positive identity matches were produced in any test frame. The HAPPY, SAD, and FEAR emotion labels were confirmed correct against the visible expressions in the test screenshots.

C. Tracking Stability

The longest observed continuous tracking session lasted 1093 seconds (approximately 18 minutes) without identity dropout or track-flip error. The servo tracking dot (visible on the dashboard video overlay) confirmed P-controller operation throughout. The 25-pixel deadzone prevented servo chatter when the face was approximately centred.

D. Latency and Throughput

Serial round-trip latency from Python send to Arduino Servo.write() was measured below 5 ms. SG90 mechanical response time (input pulse to physical position) was approximately 80–120 ms, giving a total system latency of 120–160 ms, acceptable for tracking normal head movements at 30 fps. Camera capture sustained 30 fps; InsightFace detection ran at approximately 12 fps in Thread 2 without blocking the capture stream.

Table II summarises the quantitative results.

TABLE II

System Performance Summary

Metric	Result	Condition
Recognition confidence range	61% – 83%	Indoor, frontal
Peak confidence (Gayatri)	83%	Good lighting
Max simultaneous faces	4	Crowd scene
False positive identities	0	All sessions
Max session duration	1093 s (~18 min)	Continuous tracking
Serial latency (PC → Arduino)	< 5 ms	115,200 baud USB
Total system latency	120–160 ms	Including SG90 mech.
Camera capture rate	30 fps	640×480
InsightFace detection rate	~12 fps	Thread 2, CPU only
Hardware cost (new parts)	₹3,000	Excl. laptop

VI. Discussion

The results confirm that all primary objectives were achieved. The key contribution — replacing direct pixel-to-angle mapping with a proportional controller on the error signal — is mathematically straightforward but has a large practical impact: it transforms the system's behaviour from "tracks a face at a constant angular offset" to "actively centres the detected face in the frame," which is the essential requirement for any pan-tilt tracking system. The absence of false-positive identity assignments across all test scenarios demonstrates that the 0.5 cosine similarity threshold is appropriately conservative for the enrolled database size.

Limitations observed during testing include reduced recognition confidence at oblique angles (> 30° off-axis) and under strong backlighting, where confidence dropped to the 55–65% range. The SG90 servo's 1.8 kg·cm torque and 0.1 s/60° speed limit tracking agility for fast head movements; its mechanical response (80–120 ms) dominates the total system latency. Recognition accuracy could be improved by enrolling more diverse samples (different angles, lighting conditions) per person.

The GPT-4.1-nano AI assistant provides a capability not typically found in academic tracking prototypes: natural-language querying of the attendance database, automated daily briefings, and anomaly detection. This demonstrates the feasibility of augmenting traditional rule-based surveillance systems with LLM-powered analytics at negligible additional cost via API access.

VII. Future Scope

Several directions exist for extending this work. A PID controller would add integral and derivative terms to eliminate steady-state error and reduce overshoot for fast-moving subjects. GPU acceleration via an NVIDIA Jetson Nano would push InsightFace throughput to 30+ fps and enable reliable recognition under challenging conditions. Replacing the SG90 with a brushless gimbal motor would eliminate mechanical response latency and extend the pan range. Liveness detection (blink, depth) would prevent photo-based identity spoofing. Cloud synchronisation to Firebase or AWS DynamoDB would support multi-site attendance aggregation, and an on-device OpenAI Whisper module would enable natural-language voice commands without internet dependency.

VIII. Conclusion

FaceTrack demonstrates a complete, integrated face recognition and servo tracking system on commodity hardware costing under ₹3,000. The central contribution is the identification and correction of the direct pixel-to-angle mapping error in closed-loop pan-tilt tracking systems, replaced by a mathematically correct proportional controller that guarantees face centring. The multi-threaded architecture sustains 30-fps video capture while running InsightFace at 12 fps without blocking the browser stream. Validated results show 61–83% recognition confidence, zero false positive identities, stable 18-minute tracking sessions, and sub-5 ms serial latency. The system establishes a replicable, low-cost template for smart attendance, security surveillance, and human-centred robotics applications, and demonstrates that a microcontroller costing ₹550 is entirely sufficient for servo-control tasks when computational workload is appropriately distributed.

References

[1] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," IEEE CVPR, 2001.

[2] L. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," IEEE CVPR Workshops, 2019.

[3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," IEEE CVPR, 2019.

[4] S. Patil and R. Kulkarni, "Servo-Based Face Tracking System Using Vision Algorithms," IEEE IoT and Intelligent Systems, 2020.

[5] S. I. Serengil and A. Ozpinar, "LightFace: A Hybrid Deep Face Recognition Framework," IEEE ASYU, 2020.

[6] A. Shinde and P. Kulkarni, "Smart Mirror Interface Using Interactive Vision Systems," IEEE HCI Conference, 2024.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," IEEE CVPR, 2016.

[8] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly Media, 2008.

[9] R. Szeliski, Computer Vision: Algorithms and Applications, Springer, 2022.

[10] C. M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.

[11] OpenCV Documentation. Available: https://opencv.org

[12] InsightFace Documentation. Available: https://insightface.ai

[13] Flask Documentation. Available: https://flask.palletsprojects.com

[14] Arduino Servo Library Reference. Available: https://www.arduino.cc

[15] IEEE Xplore Digital Library. Available: https://ieeexplore.ieee.org